

Modül 02: Prompt Mühendisliği, Şablonlar ve Bağlam Yönetimi

Kurumsal Sistemlerde Bir Yazılım Disiplini Olarak Promptlar

Eğitmen: Yapay Zeka Mühendisliği Eğitim Ekibi

Teknoloji Yığını: Java 25/27 (LTS), Spring Boot 3.4/4.x, Spring AI 1.0+, Ollama

1. Yazılım Bileşeni Olarak Promptlar

Kurumsal uygulamalarda prompt'ları basit String birleştirmeleriyle (`+`) oluşturmak ciddi bir anti-pattern'dir:

- Sürümlenemez ve test edilemez.
- Prompt enjeksiyonu ve sızdırma saldırılarına açıktır.
- Çok dilli desteği ve bakımını zorlaştırır.

Spring AI, prompt'ları birinci sınıf yazılım varlıklarına dönüştürür:

- `PromptTemplate` ve `TemplateEngine`.
- Rol tabanlı mesaj nesneleri (`SystemMessage`, `UserMessage`, `AssistantMessage`).
- Sabit talimatlar ile dinamik verilerin net ayrımı.

2. Prompt Anatomisi ve Mesaj Roller

```
SystemMessage: Rol, Kurallar, Çıktı Formatı  
"Sen uzman bir SQL mühendisisin. Her zaman ANSI üret."
```

```
Few-Shot Örnekleri: İstenen girdi-çıktı gösterimi  
Örnek 1: Soru -> SQL  
Örnek 2: Soru -> SQL
```

```
UserMessage: Kullanıcının güncel sorusu ve bağlam  
"2025 yılında en çok harcama yapan ilk 5 müşteriyi bul"
```

3. Dosya Tabanlı PromptTemplate Kullanımı

Prompt şablonlarını `src/main/resources/prompts/*.st` dosyalarında saklayın:

```
@Service
public class SqlGeneratorService {

    @Value("classpath:prompts/sql-generator-system.st")
    private Resource systemTemplateResource;

    public String buildPrompt(String schema, String userQuery) {
        PromptTemplate template = new PromptTemplate(systemTemplateResource);
        return template.render(Map.of(
            "schema", schema,
            "question", userQuery
        ));
    }
}
```

4. Few-Shot Örneklemeli Öğrenme

Few-shot öğrenme, modelin olasılık dağılımını hedef örneklerle göre yönlendirir:

```
public record SqlExemplar(String question, String sql, String explanation) {}

public String formatFewShotExamples(List<SqlExemplar> exemplars) {
    StringBuilder sb = new StringBuilder("Few-Shot Örnekleri:\n");
    for (var ex : exemplars) {
        sb.append("Soru: ").append(ex.question()).append("\n");
        sb.append("SQL: ").append(ex.sql()).append("\n\n");
    }
    return sb.toString();
}
```

İnce ayar (fine-tuning) gerekmeden şema uyumluluğunu en üst düzeye çıkarır!

5. Prompt Enjeksiyonu Savunması ve Temizleme

Kurumsal verileri ve sistem promptunu korumak için savunma hattı:

```
@Component
public class PromptInjectionSanitizer {
    private static final List<Pattern> ATTACK_PATTERNS = List.of(
        Pattern.compile("(?i)ignore\\s+(all\\s+)?previous\\s+instructions"),
        Pattern.compile("(?i)system\\s+prompt\\s+override")
    );

    public boolean detectInjection(String prompt) {
        return ATTACK_PATTERNS.stream().anyMatch(p -> p.matcher(prompt).find());
    }
}
```

Kullanıcı girdilerini modele göndermeden önce mutlaka güvenlik süzgecinden geçirin!

6. Ödev 02 Yönergesi ve Uygulama

"Akıllı SQL Sorgu ve Açıklama Üreteci"

1. `module-02-prompt-engineering/homework/starter` dizinini açın.
2. `SqlGeneratorService.java` dosyasını inceleyin.
3. `PromptTemplate` kullanarak `generateSqlQuery` metodunu tamamlayın.
4. Kötü niyetli girdilerde `"REJECTED"` dönecek şekilde enjeksiyon koruması ekleyin.
5. `mvn test` komutunu çalıştırarak tüm testlerin geçtiğini doğrulayın.